Context-Aware Security Feedback for AI-Generated Code in Fintech Applications: Thesis Proposal

AI coding assistants (ChatGPT, GitHub Copilot, etc.) are now commonly used to build fintech apps that handle sensitive user accounts, payments, and transactions. However, this widespread use has introduced a systemic security challenge. A large-scale analysis of 7,703 AI-attributed files from public GitHub repositories found that 12.1% contained at least one Common Weakness Enumeration (CWE)-mapped vulnerability (Schreiber & Tippe, 2025). Across these files, static analysis identified 4,241 distinct CWE instances, with important vulnerability types including:

- SQL Injection.
- OS Command Injection.
- Code Injection, and
- The use of hard-coded credentials.

Furthermore, an empirical study analyzing 733 AI-generated code snippets (produced by GitHub Copilot, CodeWhisperer, and Codeium) extracted from real GitHub repositories found that 27.3% contained security weaknesses (Fu et al., 2025).

Existing static application security testing (SAST) tools such as Semgrep can flag many of these patterns automatically. But detection alone does not necessarily explain the application's security context or how the developer should remediate the problem.

This issue is particularly significant in the context of Broken Object Level Authorization (BOLA). BOLA occurs when an application successfully authenticates a user but fails to verify whether that user is authorized to access or perform an operation on the specific object identified in an API request. In a fintech application, for example, an API may accept a valid transaction ID and execute a technically correct database query, yet still expose a vulnerability if it does not confirm that the authenticated user is permitted to access or modify the corresponding transaction. An attacker could potentially exploit this weakness by altering the transaction ID in the request to access another user's transaction data or perform an unauthorized action. This raises the central question behind this thesis:

Can an interactive security-feedback system, designed to provide contextual guidance at the point when developers review AI-generated code, improve developers' ability to identify and fix vulnerabilities compared with out-of-context static security warnings?

Statement of the Problem

AI-generated code used in fintech applications recurrently contains specific, well-documented vulnerability types, including:

- Hard-coded credentials (CWE-798/259).
- Injection flaws (CWE-89, CWE-78, CWE-94).
- Improper input validation (CWE-020), and
- Broken authorization at the API level (Broken Object Level Authorization (BOLA).

BOLA is a significant API security problem and is particularly relevant to fintech applications that expose transactions, accounts, and other financial objects through APIs. Industry reporting has estimated that BOLA vulnerabilities are involved in approximately 40% of API attacks. Recent empirical research by Kaur (2026) based on publicly disclosed bug-bounty reports has also examined BOLA prevalence, including different BOLA patterns and their severity. In the study, 84 of 107 classified reports were confirmed as in-scope BOLA cases, while action-level BOLA accounted for 41.7% of the confirmed cases. These findings establish BOLA as a recurring security problem rather than an isolated implementation mistake.

The problem becomes more significant in AI-assisted development because AI-generated code may correctly implement the functional part of an API endpoint while omitting the authorization relationship between the authenticated user and the requested object. Unlike a simple syntax error, identifying this flaw requires understanding the application's users, objects, ownership relationships, and permitted actions.

Existing static security tools such as Semgrep can detect security patterns, but their findings may not provide the application-specific context needed to understand why an otherwise valid object lookup is unauthorized. There is limited evidence on whether feedback designed specifically to expose this authorization context can measurably improve developers' ability to identify and remediate BOLA vulnerabilities in AI-generated fintech code. This study therefore investigates whether a context-aware feedback layer can provide a measurable improvement over conventional static warnings using a working fintech-style testbed and developer evaluation.

Aim

To design, build, and evaluate a developer-centered, context-aware security feedback tool that improves the identification and remediation of BOLA vulnerabilities in AI-generated fintech code, and to demonstrate this improvement against a purpose-built fintech testbed and developer user study.

Specific Objectives

1. **Identify BOLA vulnerabilities**: To establish the occurrence, key factors, and security requirements associated with BOLA in AI-generated fintech code.

2. **Design a fintech app**: To design and build a small but realistic fintech application (login, accounts, and transaction/transfer functionality) using AI coding assistants, to serve as a concrete testbed rather than isolated code snippets.

3. **Implement the work**: To implement a lightweight VS Code extension that uses Semgrep to detect selected BOLA patterns and presents contextual, developer-centered remediation feedback.
4. **Evaluate**: To test the extension against the fintech testbed and conventional static warnings, and to run a developer user study measuring whether the contextual feedback improves BOLA identification and remediation.

Methodology

Objective 1 — Identify BOLA vulnerabilities.

Review existing literature and empirical studies on BOLA, AI-generated code security, and fintech security to establish the prevalence, fintech impact, root causes, and key factors associated with BOLA. The review will be anchored to the OWASP API Security Top 10 and secure-by-design literature, including CISA's Secure-by-Design principles and secure software design principles such as fail-safe defaults and complete mediation. These principles will be used to establish what secure object-level authorization should look like in the fintech testbed. Success is measured by producing a literature-backed definition of the selected BOLA scenarios, their key contributing factors, fintech impact, and corresponding security requirements.

Objective 2 — Design a fintech app.

Build a small fintech web application (login, account view, and transaction/transfer functionality) using AI coding assistants for parts of the implementation, so the testbed's code reflects AI-generated patterns rather than hand-picked snippets. The application will include separate users and transaction objects to create realistic object-level authorization relationships. Success is a working application containing controlled BOLA scenarios, including at least one case of unauthorized access or modification of another user's transaction.

Objective 3 — Implement the work.

Build the VS Code extension: Semgrep (with existing and custom rules) as the detection engine, with a contextual feedback layer that explains the relationship between the authenticated user and the requested object, why the authorization condition is required, and how the developer can remediate the vulnerability. Run the extension against the Objective 2 testbed and record what it catches and what contextual information it adds beyond the raw static warning. Success is the extension correctly identifying the selected BOLA scenarios and providing appropriate remediation guidance.

Objective 4 — Evaluate.

Two-part evaluation:

(a) run plain Semgrep and the extension against the same testbed code side by side, and document what the extension's contextual layer adds beyond the raw warning, including explanation of the authorization relationship and remediation guidance;

(b) run a within-subjects developer user study in which participants review matched AI-generated vulnerable code under conventional static warnings and the extension's contextual feedback (order counterbalanced), measuring BOLA vulnerabilities correctly identified, remediation correctness, understanding of the authorization issue, and time taken. Success is measured directly against the aim: evidence of whether, and by how much, the context-aware feedback improves BOLA identification and remediation over conventional static warnings.

References

CISA. *Principles and approaches for security-by-design*. Available at: https://www.cisa.gov/sites/default/files/2023-04/principles_approaches_for_security-by-design-default_508_0.pdf

Ebad, S. (2022) *(PDF) Exploring how to apply secure software design principles*. Available at: https://www.researchgate.net/publication/366112299_Exploring_How_to_Apply_Secure_Software_Design_Principles

Fu, Y. *et al.* (2025) *Security weaknesses of copilot-generated code in GitHub projects: An empirical study*, *arXiv.org*. Available at: https://arxiv.org/abs/2310.02059

Kaur, B. (2026) *Broken object level authorization in the wild: An empirical taxonomy from 100+ bug bounty disclosures*, *arXiv.org*. Available at: https://arxiv.org/abs/2605.25865.

Schreiber, M. and Tippe, P. (2025) *Security vulnerabilities in AI-generated code: A large-scale analysis of public GitHub repositories*, *arXiv.org*. Available at: https://arxiv.org/abs/2510.26103